

is performed using Image-Prompt-ONNX-Conversion.py. To support multiple reference images in a single workflow, the original script was extended into Multiple-Mixed-Image-ONNX-Conversion.py, allowing several images of the same object to be used together.

During the experiments, the following objects were found to be unrecognised by the pre-trained model:

- Stapler
- Keyring
- Magnifying lens
- Coaster plate

Two or three images of each object (pic1.jpeg, pic2.jpeg, and pic3.jpeg) were captured from different angles. These images can be downloaded from the internet, taken using a mobile phone, or captured directly on a Raspberry Pi using Image-Prompt-Capture.py.

Next, Image-Prompt-Draw-Box.py is used to manually define the bounding box for each object in every reference image. Using a mouse or trackpad, a tight bounding box is drawn around each object. A portion of the generated configuration file is shown in Fig. 4.

After running Multiple-Mixed-Image-ONNX-Conversion.py, these four objects become embedded in the exported yoloe-11s-seg.onnx model. Whenever YOLOE-Run-Model.py is executed, the model recognises these objects and returns their assigned class IDs.

Within the inference program, a simple class mapping converts the numeric IDs into human-readable labels, as shown in Fig. 5.

Using this visual prompting approach, YOLOE can be rapidly customised to recognise completely new objects without conventional model retraining. This transforms a general-purpose vision model into a lightweight, task-specific object detector with minimal effort.

A helpful video demonstrating image prompting with YOLOE is available at:

```
(yoloenv) somnathbera@Somnaths-MacBook-Air yolo-code-core-electronics % python Text-Prompt-ONNX-Conversion.py
```

```
Ultralytics 8.3.223 Python-3.11.6 torch-2.9.0 CPU (Apple M1)
```

```
YOLOe-11s-seg summary (fused): 137 layers, 13,693,398 parameters, 0 gradients, 36.4 GFLOPs
```

```
PyTorch: starting from 'yoloe-11s-seg.pt' with input shape (1, 3, 640, 640) BCHW and output shape(s) ((1, 41, 8400), 32, 160, 160) (26.5 MB)
```

```
ONNX: starting export with ONNX 1.19.1 (opset 22)...
```

```
ONNX: slimming model with onnxslim 0.1.75...
```

```
ONNX: export completed successfully (1.15 s)
```

```
Model saved as 'yoloe-11s-seg.onnx' (30.7 MB)
```

Fig. 3: Authors' console output showing the successful conversion of the YOLOE-11s-Seg model to ONNX, optimised with onnxslim

```
https://www.youtube.com/watch?v=yNPwsKa52zs&t=624s
```

One limitation observed during the experiments is that text-prompted and image-prompted objects cannot currently be combined into a single ONNX model. At present, separate models must be generated for each approach. This capability may become available in a future release of YOLOE.

EFY note. The additional software suitable for laptop testing has been attached as a ZIP file, while the latest software bundle for the Raspberry Pi can be obtained from the following link:

```
https://core-electronics.com.au/attachments/uploads/yoloe-code-core-electronics.zip
```

Typical use cases of this project (aligned with YOLOE's strengths)

Since YOLOE can recognise both predefined and user-defined objects without extensive retraining, it can be adapted to many real-world scenarios. Some typical applications are outlined below.

1. Personal object identification. Personal belongings, such as a laptop charger, spectacles, a keyring, a wallet, or tools on a desk or in a workspace, can be identified without training a large custom model.

2. Assistive vision applications. Lightweight, real-time object recognition can assist visually impaired users in locating frequently used objects, such as a smartphone, glasses, a remote control, or a clock.

```
{
  "person": {
    "id": 0,
    "samples": {
      "model_pic/pic1.jpeg": [57, 88, 370, 707],
      "model_pic/pic2.jpeg": [178, 79, 530, 823],
      "model_pic/pic3.jpeg": [21, 278, 768, 626]
    }
  },
  "stapler": {
    "id": 1,
    "samples": {
      "model_pic/pic1.jpeg": [249, 719, 487, 1001],
      "model_pic/pic2.jpeg": [53, 906, 529, 1001],
      "model_pic/pic3.jpeg": [143, 654, 593, 888]
    }
  }
}
```

Fig. 4: The generated configuration file

```
CLASS_NAMES = {
    0: "magnifier",
    1: "stapler",
    2: "keyring",
    3: "coaster"
}
```

Fig. 5: Human-readable labels

3. Smart home and office automation. Automated actions can be triggered when specific objects are detected, for example, by logging device usage, monitoring misplaced items, or generating alerts when restricted objects are present.

4. Retail and inventory spot checks. Selected products or SKUs can be verified quickly using a compact, task-specific model instead of deploying a full-scale object detection system.

5. Industrial and workshop monitoring. Specific tools or components, such as staplers, magnifying lenses, or parts trays, can be